Numerical Stability and Singularities in Path-Following Methods: Solving Issue #492 of Gambit repository

Andrés Fernández Cervell

March, 2026

Introduction

This technical report details the mathematical and algorithmic refactoring carried out to resolve numerical instabilities in the path-following solvers of the Gambit repository.

The problem to be solved was implementing a robust warning system that would notify the user if a NE had not been reached and stabilize the Predictor-Corrector Method in bifurcation points.

This report will have the following structure: chapter 1 is a description of my progress on the project, the ideas I tried but failed, and how I eventually reached an optimal solution, that will be explained with detail in chapter 2. Finally, results in games that have been tested will be discussed, and potential problems that my adjustments may face under some particular parameters.

1 Progress

I spent the first few days trying to comprehend the Mathematics and its relationship with the code of the Algorithm, testing it with some games that made it fail, such as `2x2x2.nfg`, or `team_mp.txt`. In order to understand the implementation and the theory behind it, I studied "A dynamic homotopy interpretation of the logistic quantal response equilibrium correspondence" by Theodore L. Turocy.

After that, I focused on the first part of the issue: warning the user when the Nash Equilibrium has not been reached.

Even though the messages sent to the terminal were mainly programmed in file `logit.cc`, I realized that it was better to program this particular warning in `efglogit.cc` and `nfglogit.cc` to ensure architectural consistency across all Gambit interfaces. Since Gambit can be accessed via command-line tools and Python bindings, placing the check at the specialization layer ensures that any interface invoking the Logit solver will correctly report the maximum regret. Furthermore, since both files use different algorithms for computing regret, I avoid extra code that would be necessary if I had decided its implementation to be in the main function (`logit.cc`).

Following the idea proposed in the forum by "Divinesoumyadip", I found it convenient to report the lowest-regret point (or the maximum lambda point) rather than just the endpoint. In order to make that implementation in `efglogit.cc` and `nfglogit.cc` and not lose the accessibility described above, I made the following changes:

After calling `tracer.TracePath` in `LogitBehaviorSolve`, `LogitBehaviorSolveLambda`, the program checks if the goal has been achieved in the process (the aimed regret or lambda, depending on

the function). It is important to note that not necessarily the best approximation corresponds to the final iteration, so we must check every step made. This could have been solved by doing a final loop that checks in every Profile of callback. However, it is avoidable if we control the maximum-lambda and the minimum regret in the TracePath. That´s why, I added the next attributes in class `TracingCallbackFunction` (with their correspondent Get methods):

- `m_best_lambda` (initialized to `-INFINITY`)
- `m_best_regret` (initialized to `+INFINITY`)
- `m_bestProfile_lambda` and `m_bestProfile_regret` (initialized with `p_game`)

In method `AppendPoint`, these variables are updated, and used in both `LogitBehaviorSolveLambda` and `LogitBehaviorSolve` to show the best approximation using `p_observer`.

Since I did not want to make any changes in the main function located in `logit.cc`, for the reasons described above, I had to use an exit code if the equilibrium has not been reached, because otherwise, the result final line that is printed in main could lead to confusion.

Although this strategy may be a little radical, I consider it to be the optimal for two main reasons:

- It is implemented in the files that deal with Mathematics, so this warning will always occur no matter the main function we use, making it more compact.
- It is honest and clear for the user, always showing the true result, and warning if the Nash Equilibrium has not been found.

After that, I focused on the stabilization of the algorithm.

When learning about the Math of the algorithm, I paid special attention to the `NewtonStep` method, and identified that the critical line was `y[k] /= b(k,k);`. That division is unstable when `b(k,k)` is nearly zero. Therefore, my first try was adding an epsilon to the elements of the diagonal of Matrix b that were very little. That epsilon (`c_pert`) was not arbitrary, but it was computed taking into account the maximum element of the Matrix. This is the code of my idea:

```
//Maximum of the critical division not to be considered a bifurcation point.  This
    is needed to avoid numerical issues when the path gets close to a bifurcation
   point, but does not actually reach it.
  const double multiplicative_epsilon = 1.0e-6;
  const double epsilon = 1.0e-16;
//Prevents multiplication by zero when the Jacobian is zero.
   double max_diag = 0;
  for(size_t i = 1; i <= b.NumColumns(); i++) {
        max_diag = std::max(max_diag, std::abs(b(i, i)));
     }

  double c_pert = multiplicative_epsilon * max_diag + epsilon;
  for (size_t k = 1; k <= b.NumColumns(); k++) {
    for (size_t l = 1; l <= k - 1; l++) {
      y[k] -= b(l, k) * y[l];
   }
    double pivot = b(k,k);
   if(std::abs(pivot) < c_pert) {      //The step is too large, indicating we are
    close to a bifurcation point.
     //Compute the perturbation to apply to the first equation to avoid getting
   stuck at the bifurcation point.  The perturbation depends on the Jacobian
   numbers, and is capped at c_pert.
```

```
      if(pivot >= 0)
       pivot += c_pert; // Perturb the diagonal to avoid getting stuck at the
  bifurcation point.
      else
        pivot -= c_pert;
      y[k]/= pivot;

    }

    else y[k] /= b(k,k);
  }
```

Later on, I realised that `NewtonStep` was not the real problem if we made sure that on `TracePath` the data we sent to the Newton´s algorithm was already adjusted, and that what I was doing was simply adding another epsilon to the already treated.

When these modifications made no improvements on the results, I started focusing on `TracePath`, and realised:

- A perturbation was only applied to the first element of the y vector when reaching a bifurcation point, leaving out many possible cases. The problem does not necessarily lie in the first direction indicated by the Jacobian matrix, so the entire diagonal should be adjusted.
- Perturbation applied was not enough in some cases such as `team_mp`.
- When the orientation of the curve changed, it was always considered as an "Emergency Situation", meaning that, in some cases, even when the curve was being followed successfully, the program crashed. This happens because in some limit points, the orientation must inevitably change, and it is not a mistake in Newton´s method, but the nature of the game. Therefore, if we take this as an error, and start to reduce the step h, it will never not change the orientation, and the program will end unsuccessfully.

2 Final Changes

For a better comprehension of my final solution, I will explain the theory behind it:

The Predictor-Corrector algorithm aims to trace the zero-set of a specialized homotopy function $H(x, \lambda) = 0$. When the code calls `p_function(u, y)`, it evaluates this function and stores the residuals in `y`. Normally, the `NewtonStep` tries to drive these values to zero to keep us on the equilibrium path.

By applying the transformation `y[i] += c_pert`, we are no longer tracing the original equilibrium manifold, but a perturbed one defined by:

$$H_i(x, \lambda) = \delta_i \cdot \epsilon \tag{1}$$

where ϵ is the `c_pert` constant and $\delta_i \in \{1, -1\}$ is the alternating sign used for symmetry breaking.

A bifurcation is a point where two paths cross (a singularity), and the math "breaks" there because the direction to follow becomes ambiguous. By shifting our target slightly away from zero, we ensure that we never hit that exact intersection. According to Sard's Theorem, this small displacement is enough to "disconnect" the crossing branches, turning a singular intersection into two separate, smooth curves that the `NewtonStep` can easily follow without the Jacobian matrix

becoming singular. A classic example of this effect can be seen with the equation $x^2 - y^2 = 0$. In its original state, this represents two lines crossing at the origin (an "X"), which is a singularity where a solver wouldn't know which branch to follow. However, if we add a small perturbation ϵ, the equation becomes $x^2 - y^2 = \epsilon$. This transformation turns the "X" into a hyperbola with two disconnected branches ")(". By shifting our target slightly away from zero, we ensure that we never hit the exact intersection.

Changing how the perturbation is applied, in order to adjust all the directions, not just one, was my first step. If the bifurcation was caused in a direction that was not modified, the fix proposed did not solve anything. I managed to do it using an altering perturbation in every variable, causing a multidimensional symmetry breaking:

```
if (pert != 0.0) {
  for (size_t i = 1; i <= y.size(); i++) {
      // Applying perturbation to every variable
      y[i] += pert * (i % 2 == 0 ? 1.0 : -1.0);
  }
}
```

Furthermore, I changed the way to compute `pert_countdown` so that the "buffer" against the bifurcation gets longer. `pert_countdown = std::max(fabs(10.0 * h), min_pert_countdown);` Where `min_pert_countdown = 0.05`

Additionally, I increased the perturbation applied to 0.0001, making it more meaningful, and multiplied by four `c_maxIter`, up to 400, giving the algorithm more opportunities to get to the equilibrium.

Finally, I only had to eliminate two lines of the original code: `y[1] += pert`, because the perturbation now applies to the whole vector, and with negative and positive values to create "noise". `accept = false` in `omega_flip == -1.0` condition.

This way, Predictor method is only considered to have failed when:

- Distance between the current point and the curve is greater than `c_maxDist` (`dist >= c_maxDist`)
- Iterations in Corrector method are not working as expected, and we are not getting close to the objective (`contr > c_maxContr`)
- A new condition, also inspired by "Divinesoumyadipt". It is true when the new lambda we are working with is negative, but the the previous one was not. `if (u.back() < min_lambda && x.back() >= 0.0) accept = false;`

Therefore, the program will not hit a false alarm when there is a bifurcation point but it is controlled, as it happened previously, causing an endless loop that ended up in not finding the NE.

To sum up, the modifications consist of giving the algorithm a greater opportunity to recover and find the correct path by changing the constants, alternating all the directions of the curve we are following to ensure that the problematic one does change. With these two fixes, and the safeguard against negative lambda values, we can be safe not to enter in an Emergency Mode when there is a bifurcation, letting our algorithm continue as usual, unless one of three mentioned situations happen.

3 Tests and Final Results

My program has been tested with the test proposed in the ISSUE, `team_mp`, avoiding the warning, and getting the following result: `NE,0.112702,0.887298,0.887298,0.112702,0.4,0.6`

Comparison: `team_mp.txt` Execution

Original Output:

```
...
1.381734,0.5,0.5,0.5,0.5,0.111789,0.888211
1.385279,0.5,0.5,0.5,0.5,0.111262,0.888738
1.385312,0.5,0.5,0.5,0.5,0.111257,0.888743
1.385313,0.5,0.5,0.5,0.5,0.111257,0.888743
1.385313,0.5,0.5,0.5,0.5,0.111257,0.888743

WARNING: Logit solver stopped at lambda =
↪  1.38531 with regret = 0.166885 (Goal was
↪  regret = 4e-08)

Best profile found =
↪  1.385313,0.5,0.5,0.5,0.5,0.111257,0.888743
```

My Implementation:

```
...
1.381734,0.5,0.5,0.5,0.5,0.111789,0.888211
1.526637,0.5,0.5,0.5,0.5,0.0919582,0.908042
1.367054,0.500063,0.500037,0.500063...
...
(successfully traverses the bifurcation)
...
4896910.625229,0.112702,0.887298,0.887298...
5386602.323916,0.112702,0.887298,0.887298...
5925263.192472,0.112702,0.887298,0.887298...

NE,0.112702,0.887298,0.887298,0.112702,0.4,0.6
```

Furthermore, I have tested the 176 games of extensions ".efg" and ".nfg" of the folders `test/tests_games/` and `contrib/games/`.

The original implementation that is currently in Gambit failed (a Nash Equilibrium is not reached) in 39 tests in total. 19 of them happened because the test itself is designed so that the program launches an exception.

With my implementation, the results are the expected in all the tests (a Nash Equilibrium is reached), excluding 3 from the `test/test_games` and 13 from the `contrib/contrib_games` (all of them failed in the original program). Therefore, my algorithm solves 4 tests (`contrib/games/2x2x2.efg`, `contrib/games/2x2x2.nfg`,`tests/test_games/basic_extensive_game.efg`, `tests/test_games/mixed_behavior_game.efg`), and, of course, `teamp_mp.txt`. However, exploring the problem in the failed tests, they have all, except for one, something in common, the last iteration showed has at least one number that is really close to smallest positive double number accepted in IEEE 754 standards, which is around 4.94x10e-324, after which the number collapses to 0.0, and therefore the algorithm ends without success.

A possible solution to this would be adopting the use of `long double`, but changing all the variables could be a hard and risky task, that may not be worth it, considering the fact that in most cases, the solution ends up being really close to a desirable one. A list of all the failed tests and their last solution is at the end of this document.

The only test that does not reach a Nash Equilibrium, and has no small numbers in the output is `./contrib/games/bayes2a.efg`, that ends with:

```
WARNING: Logit solver stopped at lambda = 320940 with regret = 7.19914e-07 (Goal was regret =
↪  2e-07)

Best profile found = 320939.776841,0.555556,0.444444,0.4,0.6,0.4,0.6,1,0,1,0,0.111111,0.888889,0,1
↪  ,0,1,0,1,0,1,0.5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.533333,0.466667,0
↪  .5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.533333,0.466667
```

after 17 seconds of execution time, the regret is almost the one expected. Since it is a very complex

game to compute, the numbers end up being mostly noise, so the algorithm does not reach the expected regret by very small, still being a very precise approximation.

Another possible risk that my implementation may face is the fact that, since the check of the change of orientation does not create an alarm anymore, it would be possible that, in very particular situations, after a bifurcation, the Predictor-Corrector Method may start traversing the curve in an unexpected direction, for example, backwards. This is why I created the condition that lambda could not go negative after being positive, so that the program can see if the algorithm has reached a wrong branch and it has to start over.

Conclusion

Solving this issue has been a highly rewarding experience, allowing me to bridge abstract topological concepts with high-performance C++ implementation.

Appendix: Failed Tests (Hardware Limits)

Here is the list of the failed tests and their last solution:

```
TEST: ./contrib/games/2x2x2x2x2.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 1013.53 with max regret = 0.000274227 (Goal was
↪  6.838e-08)
Best profile found =
↪  1013.526036,1,1.21386e-195,0.229721,0.770279,0.630741,0.369259,0.699656,0.300344,1,2.53161e-309
------------------------------------------------------
TEST: ./contrib/games/5x4x3.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 182.939 with max regret = 0.000970009 (Goal was
↪  6.838e-08)
Best profile found = 182.938902,1,1.60192e-257,9.28077e-291,6.78725e-162,5.99387e-296,1.35627e-186
↪  ,0.387464,0.612536,1.4883e-193,0.00397391,0.996026,4.00483e-234
------------------------------------------------------
TEST: ./contrib/games/bayes2a.efg
WARNING: Logit solver stopped at lambda = 320940 with regret = 7.19914e-07 (Goal was regret =
↪  2e-07)
Best profile found = 320939.776841,0.555556,0.444444,0.4,0.6,0.4,0.6,1,0,1,0,0.111111,0.888889,0,1
↪  ,0,1,0,1,0,1,0.5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.533333,0.466667,0
↪  .5,0.5,0.333333,0.666667,0.333333,0.666667,0.533333,0.466667,0.533333,0.466667
------------------------------------------------------
TEST: ./contrib/games/bcp4.efg
WARNING: Logit solver stopped at lambda = 372.22 with regret = 0.000738337 (Goal was regret =
↪  4e-08)
Best profile found = 372.220036,7.43849e-163,4.94066e-324,1,2.22276e-162,0,1,0.249265,0.750735,9.8
↪  8131e-324,0,1,2.22276e-162,1,2.22276e-162,4.94066e-324
------------------------------------------------------
TEST: ./contrib/games/caro2.efg
WARNING: Logit solver stopped at lambda = 4.47477e+06 with regret = 6.20108e-08 (Goal was regret =
↪  4e-08)
Best profile found = 4474774.875417,0.605219,0.394781,1,0,0,1,0.535073,0.464927,0.20174,0.79826,0,
↪  1,0.333334,0.666666,0,1,0,1,0.333333,0.666667,1,0,1,0
------------------------------------------------------
TEST: ./contrib/games/cent2.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 1300.94 with max regret = 0.000187053 (Goal was
↪  1.2526e-07)
Best profile found = 1300.938472,9.02615e-314,0.902372,0.0976276,0.683027,0.316973,2.11002e-246
------------------------------------------------------
TEST: ./contrib/games/e16.efg
WARNING: Logit solver stopped at lambda = 1870.15 with regret = 0.00010243 (Goal was regret =
↪  6e-08)
Best profile found = 1870.154855,0.5,0.5,0,1,0,1,0.375,0.625,0,1,0.375,0.625,0,1,0.375,0.625,2.665
↪  48e-305,1,1,0,1,0,0.500034,0.499966,1,0,0.500034,0.499966,1,0,0.500034,0.499966
------------------------------------------------------
TEST: ./contrib/games/g2.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 122.829 with max regret = 0.00187586 (Goal was 8e-08)
Best profile found = 122.829004,0.427769,0.572231,8.77591e-108,9.57738e-311,1,2.60816e-125,5.06983
↪  e-24,0.665877,0.334123
------------------------------------------------------
TEST: ./contrib/games/g3.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 326.722 with max regret = 0.00084843 (Goal was 7e-08)
Best profile found = 326.721988,0.19958,0.80042,1,2.2727e-322,1,2.93299e-171,0.66525,0.33475
------------------------------------------------------
```

```
TEST: ./contrib/games/mixdom2.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 339.572 with max regret = 0.000477619 (Goal was 1.4e-07)
Best profile found =
↪  339.571523,1.71908e-207,0.500119,1.76101e-266,0.499881,3.01334e-222,6.23527e-296,0.4,0.6
--------------------------------------------------
TEST: ./contrib/games/poker.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 903.717 with max regret = 0.000303914 (Goal was 5e-08)
Best profile found = 903.717455,2.18445e-295,0.500608,2.17914e-295,0.499392,0.749999,0.250001
--------------------------------------------------
TEST: ./contrib/games/todd1.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 174.865 with max regret = 0.0013213 (Goal was 1.6e-07)
Best profile found = 174.864643,8.48013e-305,0.499505,8.45775e-305,0.500495,2.75976e-102,6.13614e-
↪  153,0.666665,0.333335
--------------------------------------------------
TEST: ./contrib/games/vd.efg
WARNING: Logit solver stopped at lambda = 743.137 with regret = 0.000369469 (Goal was regret =
↪  4e-08)
Best profile found = 743.137294,1.97626e-323,1,0.749631,0.250369,0.749631,0.250369,0.250369,0.7496
↪  31,0.250369,0.749631
--------------------------------------------------
TEST: ./tests/test_games/e01.efg
WARNING: Logit solver stopped at lambda = 743 with regret = 8.97971e-06 (Goal was regret = 4e-08)
Best profile found = 743.000294,1,1.72923e-322,1,3.50787e-322,0.000951849,0.999048
--------------------------------------------------
TEST: ./tests/test_games/e02.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 699.722 with max regret = 2.15777e-05 (Goal was 3e-08)
Best profile found = 699.722216,0.999978,1.30165e-304,2.19137e-05,0.507666,0.492334
--------------------------------------------------
TEST: ./tests/test_games/mixed_strategy.nfg
Warning: corrector failed to converge after 400 iterations; rejecting predictor step.
WARNING: Logit solver stopped at lambda = 699.722 with max regret = 2.15777e-05 (Goal was 3e-08)
Best profile found = 699.722216,0.999978,1.30165e-304,2.19137e-05,0.507666,0.492334
--------------------------------------------------
```